Group 05 - Team Newbie

Online booking and management system
Software Architecture Document

Version 3.0

Revision History

Date	Version	Description	Author
20/11/2025	1.0	Section 1, 2, 3 and Review & Analytics subsystem class diagram	Phạm Trần Thanh Phong
21/11/2025	1.0	Section 4.1; 4.2 and Payment & Disputes subsystem subsystem class diagram	Nguyễn Văn Phước
21/11/2025	1.0	Section 4.3 and Booking & Viewing subsystem subsystem class diagram	Huỳnh Đăng Khoa
21/11/2025	1.0	Section 4.4 and Room Management & Search subsystem subsystem class diagram	Hà Công Thuận
21/11/2025	1.0	Section 4.5 and Authentication & User Management subsystem subsystem class diagram	Nguyễn Quốc Lộc
12/12/2025	2.0	Section 5: Draw a deployment diagram and write a description	Nguyễn Quốc Lộc
12/12/2025	2.0	Write a description for the entire class diagram	Huỳnh Đăng Khoa
12/12/2025	2.0	Section 6.1: Client-side Project Structure	Nguyễn Văn Phước
12/12/2025	2.0	Section 6.2: Server-side Project Structure	Hà Công Thuận
12/12/2025	2.0	Write a description for the entire diagram except for the class diagram.	Phạm Trần Thanh Phong
20/12/2025	3.0	Class Diagram for frontend	Nguyễn Quốc Lộc
26/12/2025	3.0	Update Authentication & User Management subsystem diagram	Nguyễn Văn Phước
30/12/2025	3.0	Update Booking subsystem subsystem class diagram	Phạm Trần Thanh Phong
1/1/2026	3.0	Update Payment & Disputes subsystem class diagram	Huỳnh Đăng Khoa
4/1/2026	3.0	Update Room Management & Search and AI subsystem class diagram	Nguyễn Quốc Lộc
4/1/2026	3.0	Update Review & Analytics subsystem class diagram	Hà Công Thuận
4/1/2026	3.0	Update Use case model	Nguyễn Quốc Lộc
10/1/2026	3.0	Update deployment diagram	Nguyễn Quốc Lộc

Table of Contents

1. Introduction **5**
1.1 Purpose 5
1.2 Scope 5
1.3 Definitions, Acronyms, and Abbreviations 5
1.4 References 5
2. Architectural Goals and Constraints **5**
2.1 Platform Technology 5
2.2 Performance Requirements 5
2.3 Security Requirements 5
2.4 Environment & Compatibility 5
3. Use-Case Model **5**
4. Logical View **6**
4.1 Tech Stack 6
4.2 High-Level Container Architecture 7
4.3 High-Level Package Structure Diagram 7
4.4 Component: Frontend 8
4.4.1 Package Diagram 8
4.4.2 Responsibility 9
4.4.3 Technologies 9
4.4.4 Connections with other components 9
4.4.5 Flow 9
4.4.6 Class Diagram 9
4.5 Component: Backend 9
4.5.1 Package Diagram 9
4.5.2 Responsibility 10
4.5.3 Technologies 10
4.6.4 Connections with other components 10
4.5.5 Flow 10
4.5.6 Class Diagram 10
Authentication & User Management subsystem: 11
Room Management & Search and AI subsystem: 11
Booking subsystem: 12
Payment & Disputes subsystem: 13
Review & Analytics subsystem: 14
5. Deployment **15**
5.1 Architecture Overview 15
5.2 Deployment Nodes 15
5.2.1 Client Tier (User Environment) 15
5.2.2 Web Server Tier (Netlify) 15

5.2.3 . Application Server Tier (Koyeb) 16
5.2.4 Data Tier (Neon) 16
5.2.5 . External Systems Integration 16
6. Implementation View 16
6.1 Client-side Project Structure 16
6.2 Server-side Project Structure 17

Software Architecture Document

1. Introduction

1.1 Purpose

This document describes the architecture of the Online Booking and Management System and defines the major architectural decisions. It ensures that developers and stakeholders share a common understanding of the system and provides guidance for implementation according to the specified requirements.

1.2 Scope

This document covers the high-level architecture of the Online Booking and Management System, including architectural styles, major components, and interactions.

1.3 Definitions, Acronyms, and Abbreviations

- API: Application Programming Interface.
- Host: User posts and manages rental rooms.
- Renter: User searches and rents rooms (Students, working people)
- Admin: System administrator

1.4 References

- *Vision Document* - *Online booking and management system* - Version 1.0

2. Architectural Goals and Constraints

2.1 Platform Technology

- **Backend:** Node.js 20.x, NestJS Framework
- **Database:** Neon PostgreSQL (Serverless PostgreSQL)
- **Frontend Deployment:** Netlify
- **Backend Deployment:** Koyeb

2.2 Performance Requirements

- The system handles ≥ 200 concurrent users.
- Page load time ≤ 3.5 seconds under standard network conditions.

2.3 Security Requirements

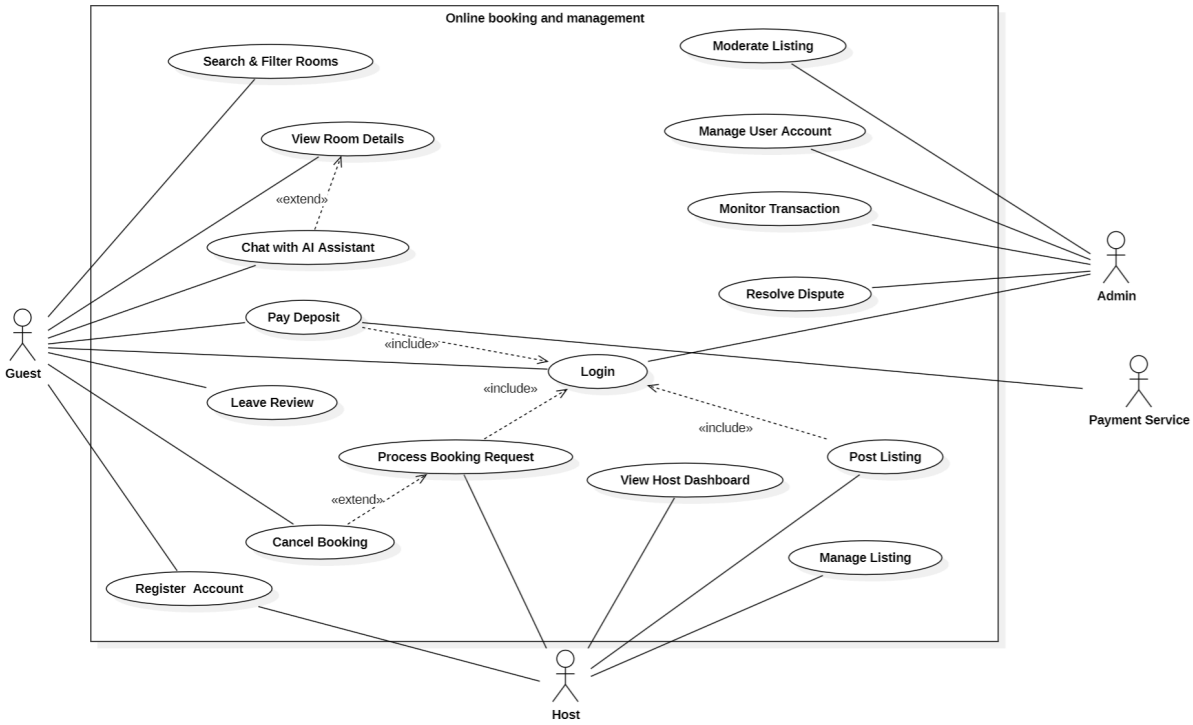
- Enforce HTTPS/TLS for all communication.
- Authenticate users with JWT (JSON Web Tokens).

2.4 Environment & Compatibility

- Application environment: Web application
- Responsive interface on Mobile / Tablet / Desktop.
- Compatible with browsers: Chrome, Firefox, Edge, Safari.
- Requires internet connection

3. Use-Case Model

- The diagram illustrates the business functions of the system with the interaction of three main actors: Guest, Host, and Admin, along with external payment services.
- Specifically, the system focuses on the process of Guest searching, using AI chatbot, and booking; Host posting listings and managing rental requests; and Admin performing content moderation and dispute resolution to ensure smooth operation.



4. Logical View

4.1 Tech Stack

- Frontend (Client-side): TypeScript + Next.js, building modern user interfaces.
- Backend (Server-side): Node.js 20.x + NestJS, handling business logic and providing API.
- Database: PostgreSQL on Neon Cloud, storing system data.
- Infrastructure: Backend deployed on Koyeb Cloud, Frontend deployed on Netlify.

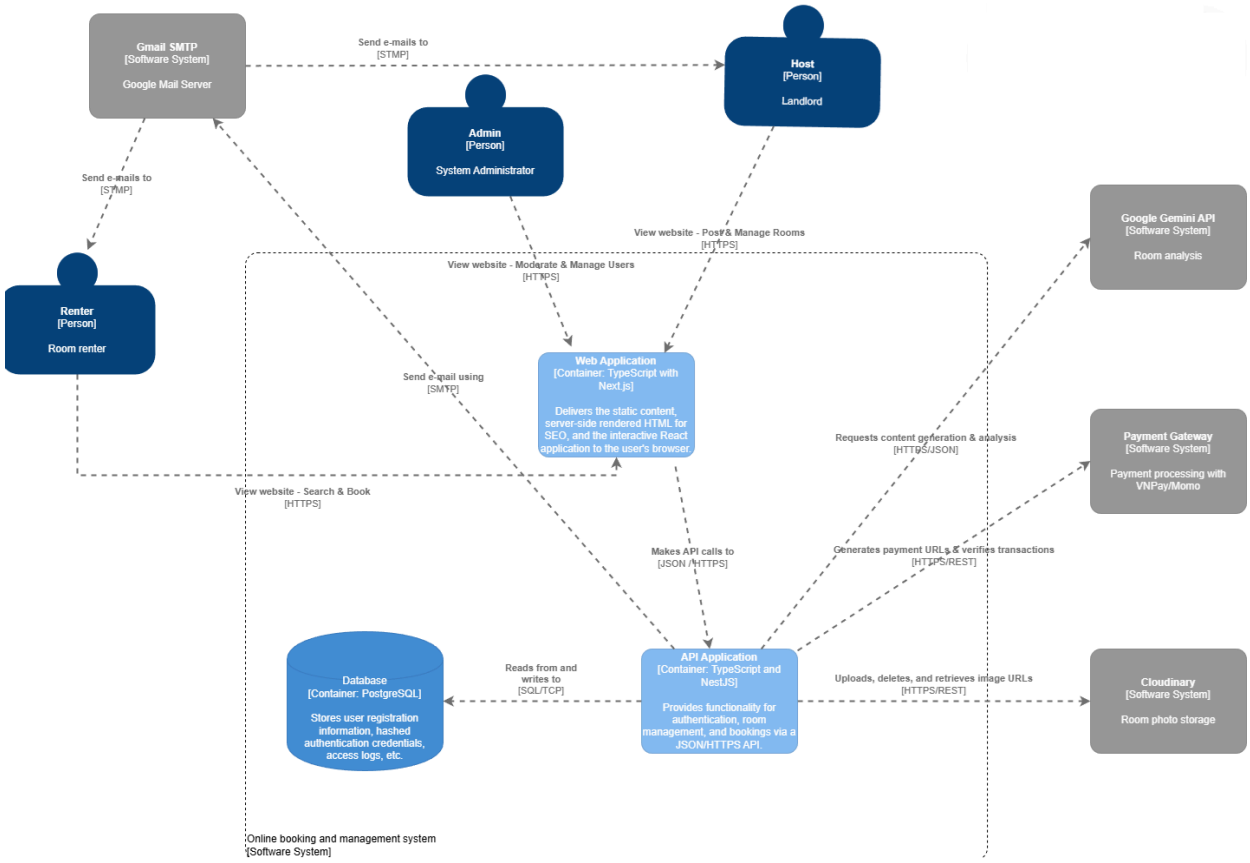
- Development Tools: GitHub source code management.

4.2 High-Level Container Architecture

The diagram illustrating the system's technical architecture is divided into two main blocks:

- **Application Block:** Consists of a Web Application (Next.js) providing an interactive interface for users (Renter, Host, Admin) and an API Application (NestJS) handling the central business logic and connecting to the PostgreSQL database.
- **Integration Block:** The API layer is responsible for communicating with external services including Google Gemini (AI analytics), Cloudinary (image storage), Payment Gateway (VNPay/Momo payments), and Gmail SMTP (email sending).

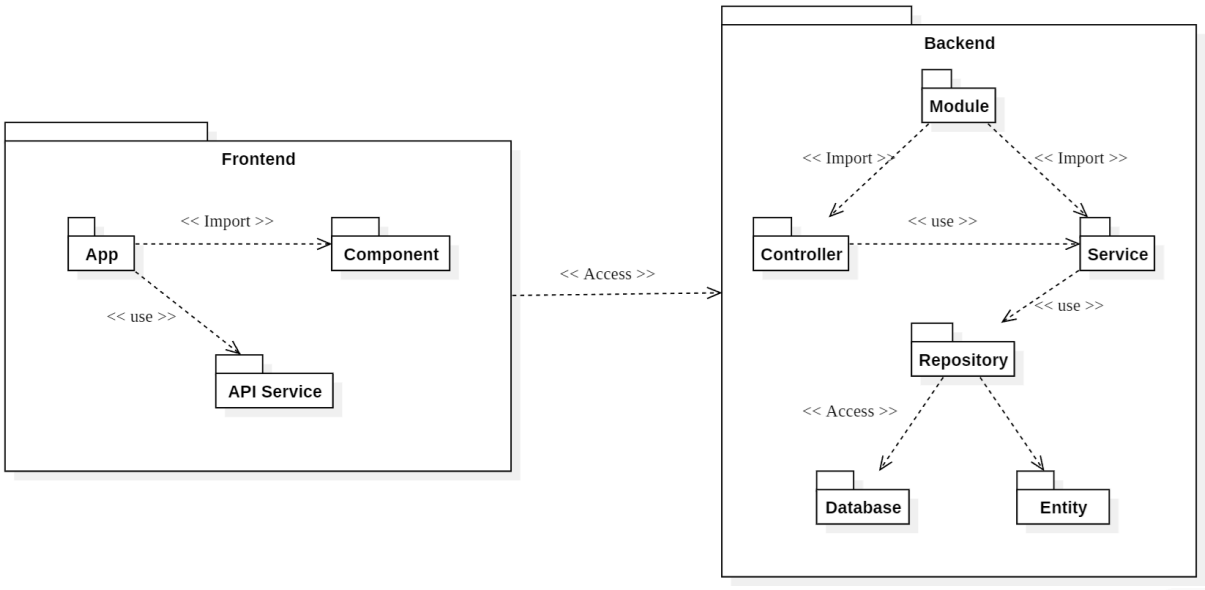
The diagram below illustrates the system's Container architecture, including web applications, APIs, databases, and integrations with external systems: Container diagram



4.3 High-Level Package Structure Diagram

The diagram illustrates the organizational structure of the system's source code according to a separate Client-Server model:

- **Frontend:** Organized in a Component-based manner. The App acts as the central hub, managing the interface Components and using API Services to send requests to the Backend.
- **Backend:** The Controller receives the request, forwards it to the Service for business logic processing, and finally, the Repository is responsible for directly interacting with the Database and Entity.

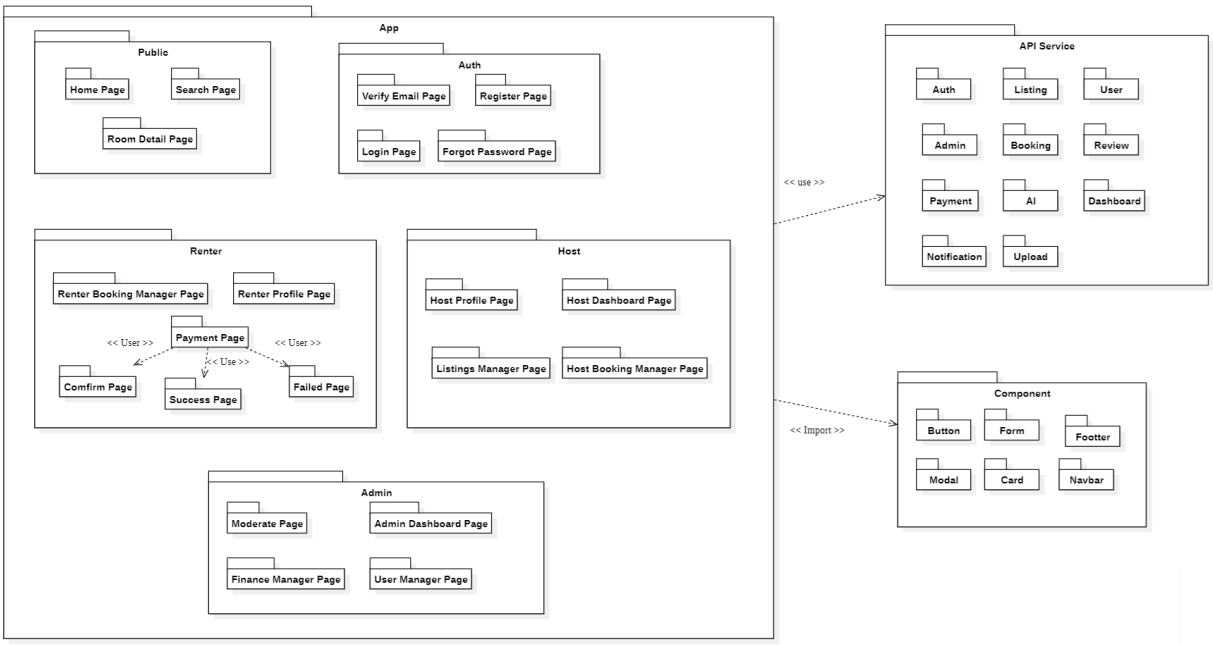


4.4 Component: Frontend

4.4.1 Package Diagram

The diagram illustrates how the Frontend source code is organized into three functional layers for optimized management:

- App: Organized according to user roles (Public, Renter, Host, Admin, Auth).
- API Service: Acts as an intermediary layer connecting to the Backend. Files are broken down by business logic (Booking, Payment, AI, etc.).
- Component: Contains common user interface elements (Buttons, Navbars, Modals, etc.).



4.4.2 Responsibility

Frontend component acts as the user interface layer (Client-side), responsible for direct interaction with the end users (Students, Hosts, Administrators). The internal architecture is divided into specialized sub-packages:

- **App & Component:** Responsible for displaying the interface (UI) and handling the user experience (UX). Components are designed as reusable modules to build pages in the App.
- **API Service:** data processing layer, completely separating data retrieval from the UI display logic.

4.4.3 Technologies

TypeScript & Next.js

4.4.4 Connections with other components

- **With Backend:** Connects to the Application Server via **HTTPS requests** to send user actions and retrieve data.
- **With End-User:** Interacts directly with the user through the **Web Browser**.

4.4.5 Flow

Frontend sends request via API Service → Backend Controller processes → Service executes the operation → Repository retrieves Database → returns data to Frontend.

4.4.6 Class Diagram

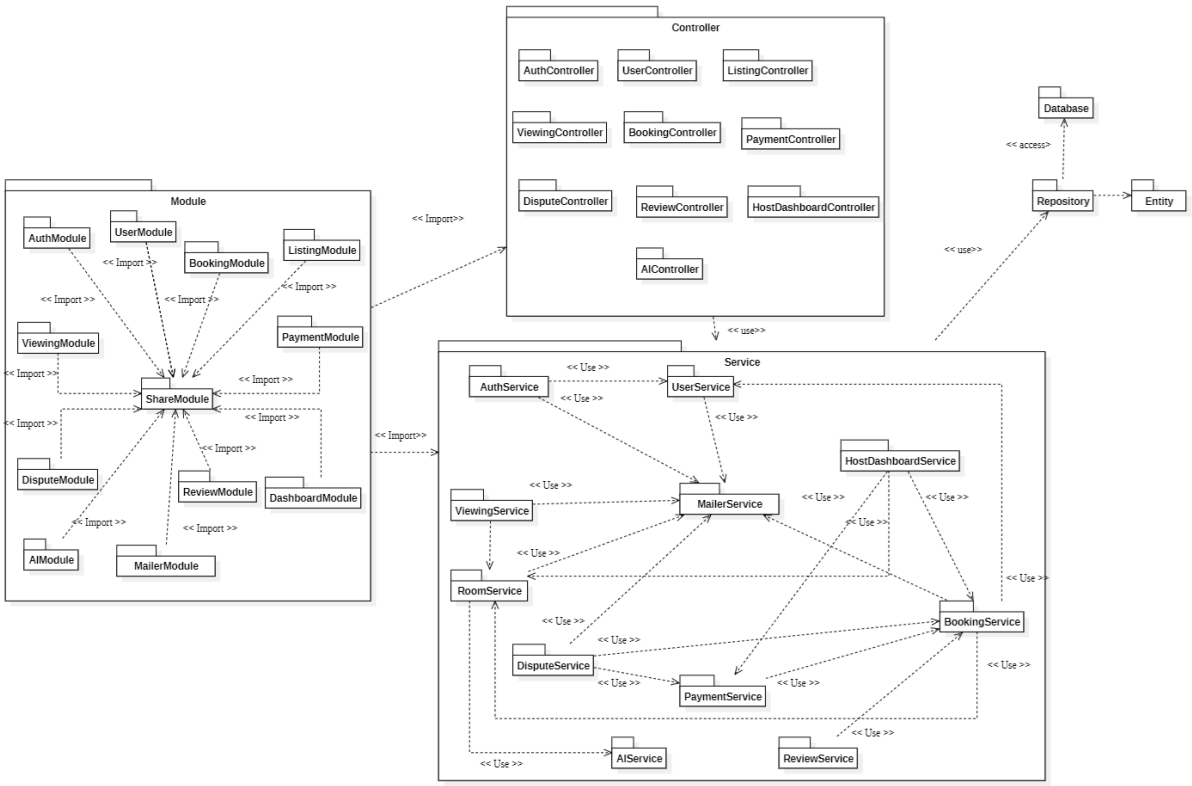
Pages and **Components** in Next.js are implemented as functional components, using functional programming.

4.5 Component: Backend

4.5.1 Package Diagram

The diagram illustrates the Backend architecture organized according to the Layered Architecture model:

- **Module Layer:** Encapsulates source code according to business functionality (Auth, User, Booking...). ShareModules play a central role, providing shared utilities for the entire system.
- **Controller Layer:** Receives requests from clients and directs responses.
- **Service Layer**: Handles business logic.
- **Data Layer (Repository & Entity):** Responsible for direct interaction with the database, ensuring complete separation of data access from business logic.



4.5.2 Responsibility

Backend component is responsible for handling all the business logic of the internal system(Server-side). The internal architecture is divided into specialized sub-packages:

- **Module:** Plays the role of configuring and managing dependencies. It performs import **Controller** and **Service** into the application.
- **Controller:** communication layer between client and backend. It receives HTTPS requests from the client, validates input data and forwards these requests to Service to perform business processing.
- **Service:** where business rules are processed before requesting to store or retrieve data.
- **Repository & Entity:** Responsible for the Data Access Layer. Repository performs direct operations with the Database, while Entity defines the data structure mapped from the database.

4.5.3 Technologies

TypeScript & NestJS

4.6.4 Connections with other components

- **With Frontend:** Exposes **RESTful API endpoints** over HTTPS to receive requests and return JSON responses.
- **With Database:** Establishes a persistent TCP/IP connection to the Relational Database via the **TypeORM** framework.
- **With External Services:** Connects to third-party APIs (Google, MoMo, VNPay, Gemini AI) via HTTP requests.

4.5.5 Flow

Controller receives & validates request → Service processes business logic → Repository interacts with Entity to access Database → Database returns data → Controller sends response back to Client.

4.5.6 Class Diagram

The entire system is divided into subsystems:

Authentication & User Management subsystem:

Dto Classes

- RegisterDto, LoginDto, UpdateProfileDto, ResetPasswordDto, CreateUserDto are data structures that help manage type and standardize input and output type for functions.

Service

- AuthService helps manage authentication and authorization. Communicate with UserService to get users information and perform authentication and authorization activity.
- UserService communicate with database and manages users

Enumeration

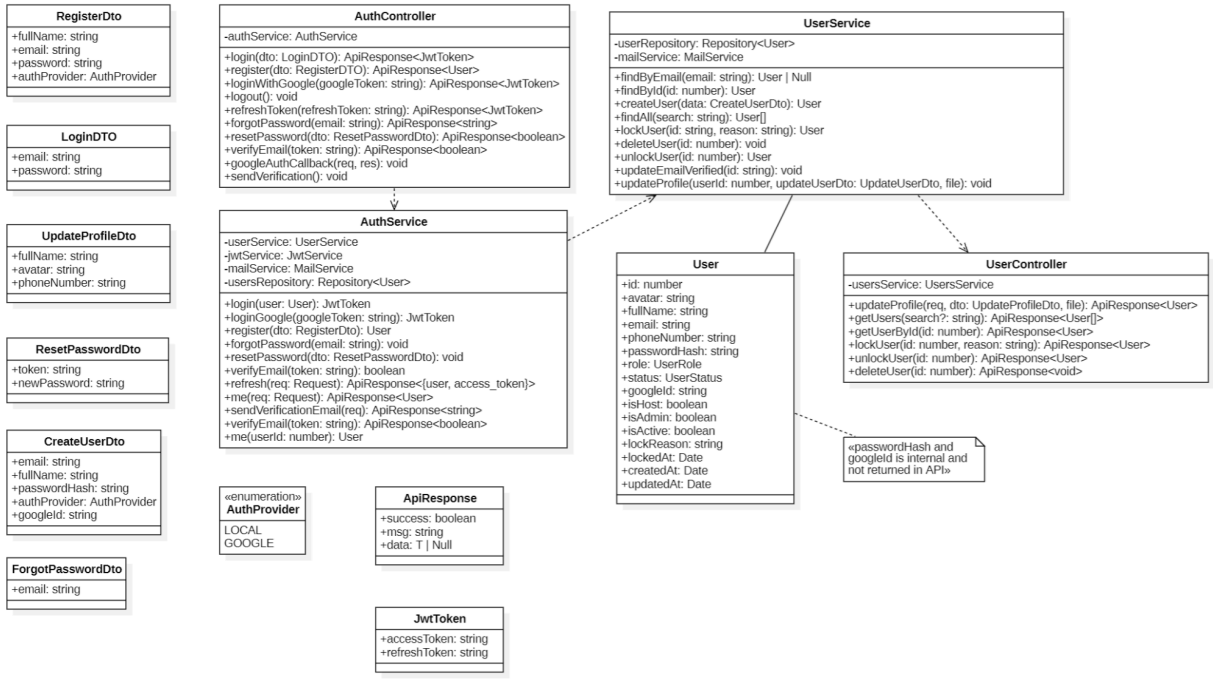
- AuthProvider are constraints defined by the developer.

Entities

- Users define a user table in the database and user type for easier programming. VerificationToke defines a verification table in the database. Use it to verify a token.
- ApiResponse standard response format.
- JwtToken manages tokens like refresh and access token.

Controller

- AuthController takes responsibility for endpoints like login, register, logout, reset password and forgot password and verifies email.
- UserController helps users manage their profile.



Room Management & Search and AI subsystem:

Dto classes

- CreateRoomDto, FilterRoommDto, ModerateDto are data structures that help manage type and standardize input and output type for functions.

Service

- RoomService communicates with databases and manages rooms.
- AiService will handle AI (gemini) communication.

Enumeration

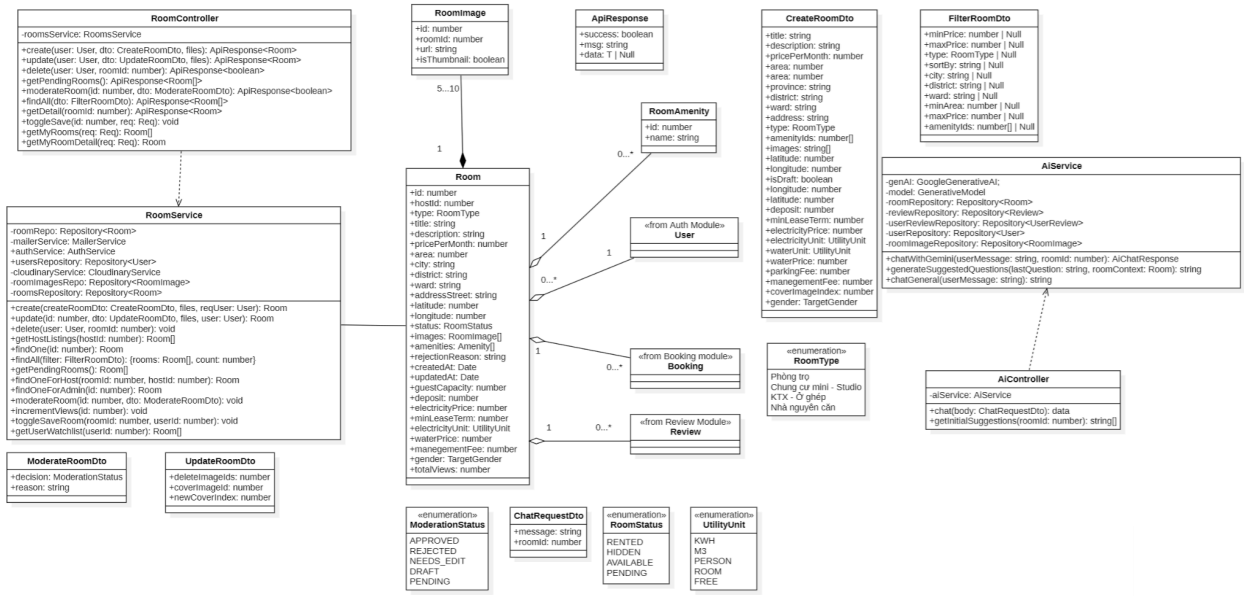
- RoomStatus, RoomType are constraints defined by the developer.

Entities

- Room, RoomImage, RoomAmenity are used to define tables in the database.

Controller

- RoomController takes responsibility for the endpoint for rooms service like getting, creating, updating, deleting



Booking subsystem:

Dto classes

- CreateBookingDto, ProcessBookingDto are data structures used to standardize input and output for booking.

Service

- BookingService communicates with repositories and performs the business logic for booking operations.

Enumeration

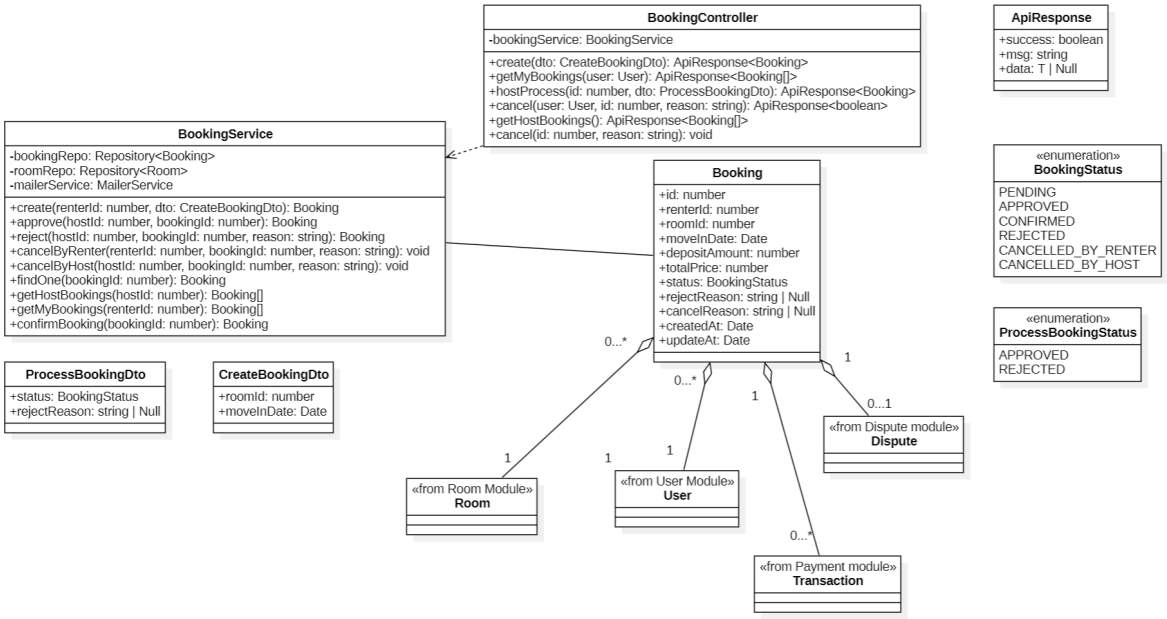
- BookingStatus, ProcessBookingStatus are constraint sets defined by the developer to ensure that bookings.

Entities

- Booking represents a full booking record linked to a specific room and renter

Controller

- BookingController handles API endpoints related to booking activities such as creating bookings, retrieving user bookings, approving or rejecting bookings, and processing cancellations



Payment & Disputes subsystem:

Dto Classes

- CreatePaymentUrlDto defines the information required to initiate a payment request
- PaymentCallbackDto represents the payload returned by an external payment gateway after a payment attempt.
- TransactionFilterDto used to filter transaction history when retrieving records
- CreateDisputeDto defines the data a user must provide when opening a dispute for a booking.
- ResolveDisputeDto used by an admin to resolve a dispute

Services

- PaymentService handles all payment-related workflows: creating payment URLs, validating gateway callbacks, updating transaction status, issuing refunds through integrated payment gateways
- DisputeService manages the lifecycle of disputes: creating disputes for booking issues, retrieving disputes pending review, resolving disputes and triggering refunds where required

Controllers

- PaymentController manages payment operations like creating deposit URLs, receiving and validating callback notifications from payment gateways, retrieving filtered transaction lists
- DisputeController provides endpoints for creating disputes, retrieving pending disputes for admin review, resolving disputes with an admin decision

Enumerations

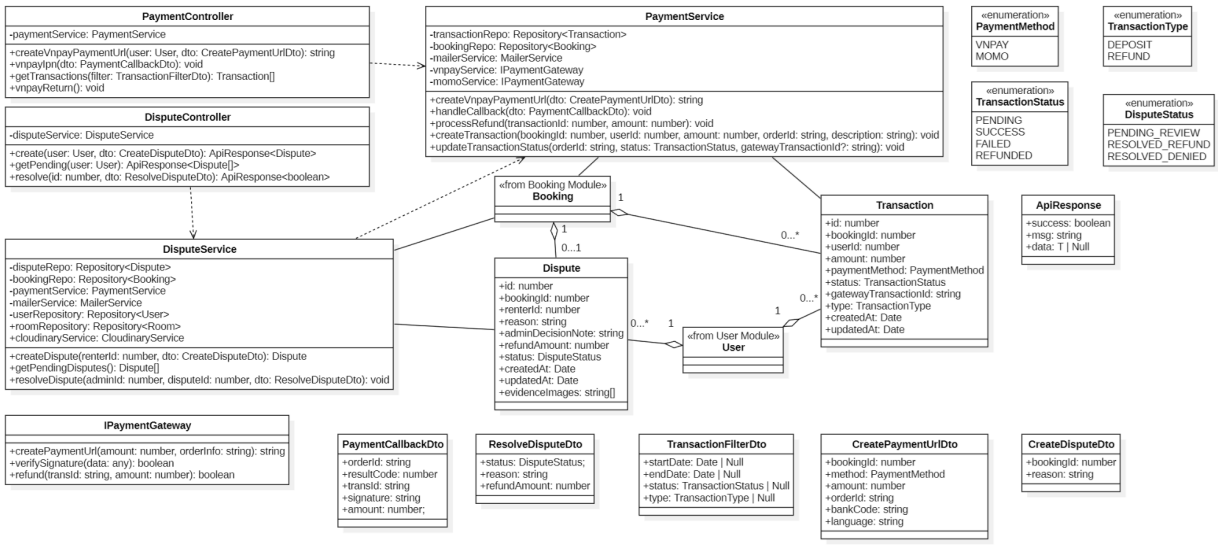
- TransactionStatus, TransactionType, PaymentMethod, DisputeStatus ensure workflows remain in valid states and prevent invalid status transitions.

Entities

- Transaction represents all financial activity linked to a booking.
- Dispute represents a user's challenge to a booking outcome.

Interface

- IPaymentGateway defines the standard operations all payment providers must implement: generate payment URLs, verify callback signatures, process refunds.



Review & Analytics subsystem:

DTO Classes

- **Dashboard DTOs:** (`InventoryStats`, `CashflowStats`, `ConversionFunnel`, `TopRoom`, `RecentTransaction`) Encapsulate aggregated KPI metrics and chart data for host analytics.
- **Review DTOs:** (`CreateReview`, `UpdateReview`, `UserReviewResponse`) Define the data structures for submitting, updating, and displaying reviews.

Services

- **HostDashboardService:** Aggregates data from Transaction, Booking, and Room repositories to calculate business performance.
- **ReviewService:** Manages room-level review logic and rating averages.
- **UserReviewService:** Handles user-to-user feedback and updates overall host ratings.

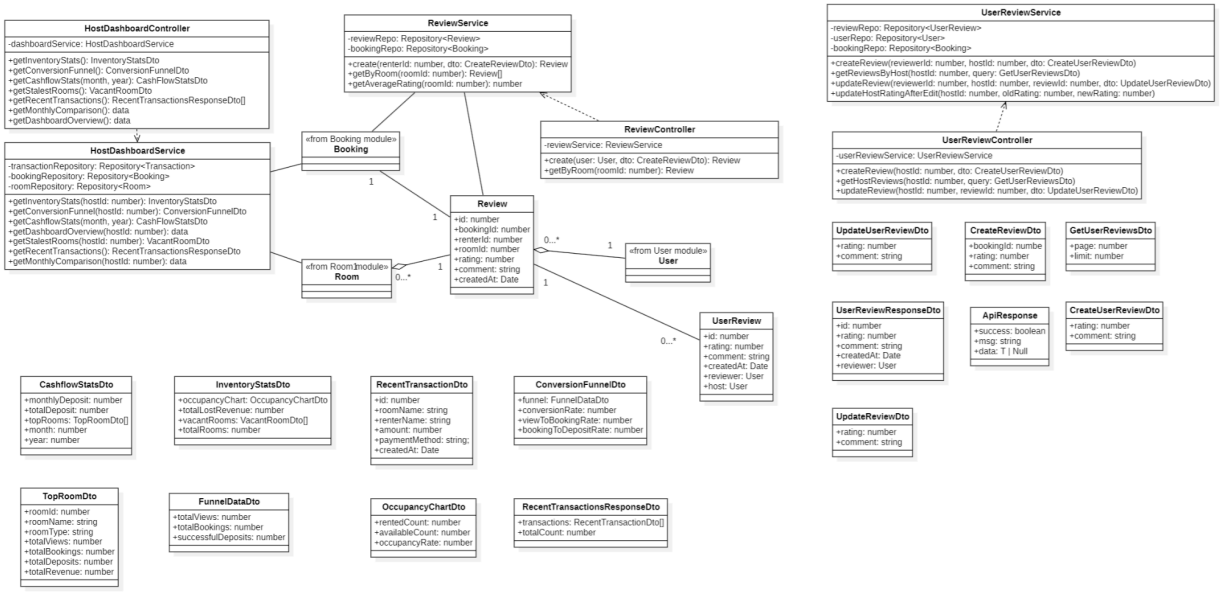
Controllers

- **HostDashboardController:** Provides API endpoints for host statistics, revenue charts, and room performance comparisons.
- **ReviewController:** Manages room review operations for renters.

- **UserReviewController:** Handles API requests for viewing, creating, and updating user-specific reviews.

Entities

- **Review:** Stores renter feedback for a specific booking and room.
- **UserReview:** Manages direct ratings and comments between users (e.g., Guest rating a Host).



5. Deployment

5.1 Architecture Overview

The system is designed and deployed using a Cloud-Native model, leveraging PaaS (Platform as a Service) and Serverless technologies to ensure scalability, high availability, and optimized operating costs. The physical architecture is divided into four distinct layers: Client, Web Server, Application Server, and Database Server.

5.2 Deployment Nodes

5.2.1 Client Tier (User Environment)

- **Execution Environment:** Cross-platform endpoints including Desktop (Windows/macOS/Linux) and Mobile (iOS/Android).
- **Software Component:** Web Browser responsible for loading and executing the client-side source code bundle (Next.js Client Bundle).
- **Protocol:** Communicates with the Web Server and App Server layers via the secure HTTPS protocol.

5.2.2 Web Server Tier (Netlify)

- **Infrastructure:** Uses the Netlify platform with a global content delivery network (CDN Edge Node).
- **Role:** Stores and distributes static resources (Static HTML, CSS, JavaScript Bundles) and public assets.
- **Configuration:** The deployment process is controlled by the netlify.toml specification file.

5.2.3 . Application Server Tier (Koyeb)

- **Infrastructure:** Uses the Koyeb Platform, providing a Node.js Runtime environment.
- **Software Components:** Contains compiled source code (dist/main.js), dependent libraries (node_modules), and environment variables (.env).
- **Business Logic:** The RoomRentalBusinessLogic component acts as the central processing unit for all API requests from clients.
- **Communication:** Provides RESTful APIs to clients via HTTPS and connects to the database via TCP/IP.

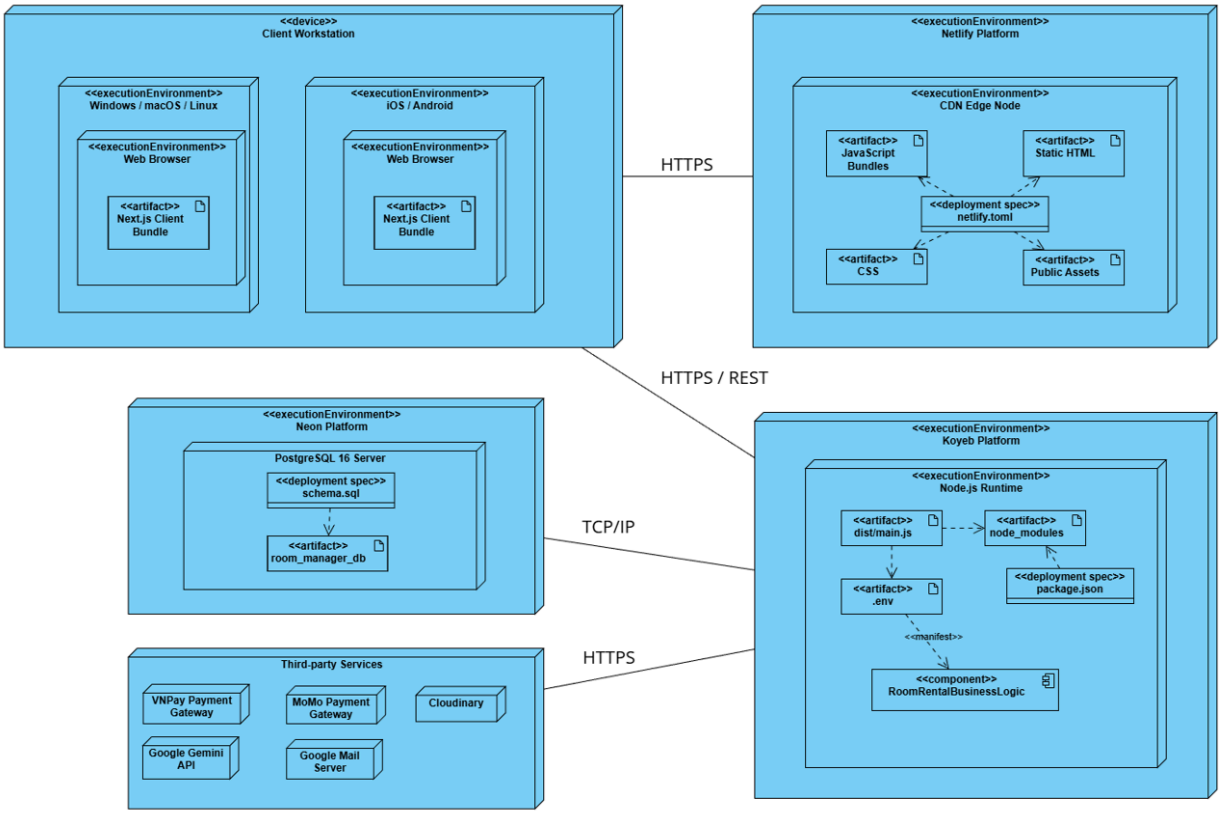
5.2.4 Data Tier (Neon)

- **Infrastructure:** Neon Platform service (Serverless Postgres).
- **Components:** PostgreSQL database server (room_manager_db).
- **Data Structures:** Initialized and managed through the schema.sql specification file.

5.2.5 . External Systems Integration

The system connects with third-party services via HTTPS protocol to extend functionality without increasing the complexity of the core infrastructure:

- **Electronic payments**: VNPay Payment Gateway, MoMo Payment Gateway.
- **Cloud storage:** Cloudinary (image/media management).
- **AI & Email services**: Google Gemini API (data analysis) and Google Mail Server (SMTP).



6. Implementation View

6.1 Client-side Project Structure

Description: The client-side application uses the Next.js App Router framework. The project structure separates the

user interface, logic, and data handling:

- **Routing & Views (src/app):** Handles URL routing and page rendering, segregated by user roles (Guest, Renter, Host, Admin).
- **Business Logic & State (src/hooks, src/contexts):** Manages application state and business logic separately from UI components.
- **Data Access Layer (src/services):** Handles HTTP communication and API requests.

```
📂 RoomRental-Frontend
├── 📂 public                # Stores static assets (Images, Fonts, Icons)
├── 📂 src
│   ├── 📂 app               # App Router: Routing and UI Pages
│   │   ├── 📂 login         # Authentication route
│   │   ├── 📂 register      # Authentication route
│   │   ├── 📂 dasboard      # Dashboard host and renter
│   │   ├── 📂 admin         # Administrator features module
│   │   ├── 📂 room          # Management room
│   │   └── 📂 search        # Search room
│   ├── 📂 components        # Shared reusable UI components
│   ├── 📂 contexts          # Global state management
│   ├── 📂 hooks             # Custom React hooks
│   ├── 📂 lib               # Utility libraries and configurations
│   ├── 📂 services          # API integration services
│   └── 📂 types             # TypeScript type definitions
├── 📜 .env                  # Environment variables
├── 📜 next.config.ts        # Next.js configuration file
├── 📜 package.json          # Project dependencies and scripts
└── 📜 tsconfig.json         # TypeScript configuration file
```

6.2 Server-side Project Structure

Description: The server-side application uses the NestJS framework. The project follows a modular structure where code is organized by feature:

- **Modular Organization (src/modules):** Groups related features (e.g., Auth, Booking, Payment) into independent modules.
- **Controllers:** Handles incoming HTTP requests and validates input data.
- **Services:** Executes business logic.
- **Entities:** Defines database schemas and interacts with the database

```
📂 RoomRental-Backend
├── 📂 dist                          # Compiled production code
├── 📂 src
│   ├── 📂 common                    # Shared resources (Guards, Interceptors, Filters)
│   ├── 📂 config                    # System configurations (Database, JWT, Mailer)
│   ├── 📂 database                  # Database migrations and seeding scripts
│   ├── 📂 modules                   # Feature-based modules
│   │   ├── 📂 booking               # [Representative Module] Example structure
│   │   │   ├── 📂 dto               # Data Transfer Objects
│   │   │   ├── 📂 entities          # Database entities
│   │   │   ├── 📜 *.controller.ts   # Request handlers
│   │   │   ├── 📜 *.service.ts      # Business logic
│   │   │   └── 📜 *.module.ts       # Module definition
│   │   ├── 📂 auth                  # (Structure similar to booking module)
│   │   ├── 📂 user                  # (Structure similar to booking module)
│   │   ├── 📂 room                  # (Structure similar to booking module)
│   │   ├── 📂 payment               # (Structure similar to booking module)
│   │   ├── 📂 ai                    # (Structure similar to booking module)
│   │   ├── 📂 dashboard             # (Structure similar to booking module)
│   │   ├── 📂 dispute               # (Structure similar to booking module)
│   │   ├── 📂 mailer                # Service configuration send email
│   │   ├── 📂 cloudiary             # Service configuration upload image
│   │   ├── 📂 review                # (Structure similar to booking module)
│   │   ├── 📂 user-review           # (Structure similar to booking module)
│   │   ├── 📂 share                 # General configuration
│   ├── 📜 app.module.ts             # Root application module
│   └── 📜 main.ts                   # Application entry point
├── 📂 test                          # End-to-end (E2E) testing files
├── 📜 .env                          # Environment variables
├── 📜 nest-cli.json                 # NestJS CLI configuration
├── 📜 package.json                  # Project dependencies and scripts
└── 📜 tsconfig.json                 # TypeScript configuration file
```